

MODULE ALGORITHMIQUE ET PROGRAMMATION C - II

LISTES CHAÎNÉES

©B. DERDOURI

CM- Listes Chaînées Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

I

PLAN

- But, Introduction , Description
- Représentation chaînée, Définition itérative et récursive, Exemple
- Implémentation chaînée, Primitives de bases
- Algorithmes de Parcours (affichage, longueur)
- Algorithmes de recherches par position et par valeur
- Algorithmes d'insertion : tête, queue, ..
- Algorithmes de suppression
- Avantages et inconvénients

©B. DERDOURI

CM Listes Chaînées Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

2

STRUCTURES DE DONNÉES DYNAMIQUES

But

- Gérer un ensemble fini d'éléments dont le nombre n'est pas fixé à priori.
- Les éléments de cet ensemble peuvent être de différents types :
 - Type simples : Entiers, Réels, Caractères, etc.
 - Type structurés : tableaux, Chaînes de caractères, Structures, etc.
- On ne s'intéresse pas aux éléments de l'ensemble en question mais aux opérations que l'on effectue sur cet ensemble, indépendamment de la nature des éléments de l'ensemble.

STRUCTURES DE DONNÉES DYNAMIQUES

Introduction

- Lorsque l'on doit stocker un ensemble de données identiques, les tableaux se révèlent intéressants.
- Cependant, ils présentent quelques inconvénients majeurs :
 - difficilement extensibles
 - insertion (resp suppression) classée n'est possible que par un décalage de tous les éléments.
- Les **listes chaînées** permettent de pallier ces inconvénients.

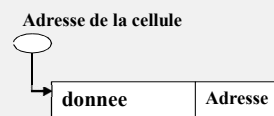
DESCRIPTION D'UNE LISTE CHAÎNÉE

- Une liste chaînée est une structure de données dans laquelle les éléments sont rangés linéairement.
- L'ordre des éléments dans une liste chaînée est déterminé par une adresse dans chaque élément.

REPRÉSENTATION CHAÎNÉE

- Dans une représentation chaînée, chaque élément indique l'adresse de l'élément suivant : On aura donc un couple, composé d'une donnée (information) et d'une adresse, que nous appellerons cellule (ou maillon).

- Une cellule est connue par son adresse :



Adresse est un pointeur vers une autre cellule (élément) ou une marque de fin s'il n'y a pas d'éléments successeur (marque NULL en C).

REMARQUE

- En allocation dynamique à la demande, les différents éléments sont dispersés en mémoire centrale. L'espace est réservé au fur et à mesure des créations d'éléments.
- La seule limite est la taille de la mémoire centrale de l'ordinateur ou l'espace mémoire allouée au processus.

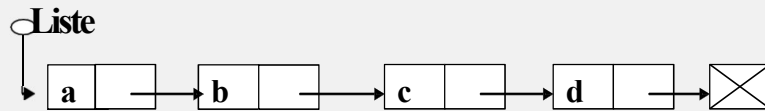
DÉFINITIONS D'UNE LISTE CHAÎNÉE

Définition itérative

- Une liste chaînée est constituée d'un ensemble fini de cellules (maillons) chaînées entre elles.
- L'adresse de la première cellule est l'adresse de la liste.
- Autrement dit, une liste chaînée est définie par l'adresse de sa première cellule.

EXEMPLE D'UNE LISTE CHAÎNÉE

- La liste linéaire [a, b, c, d] est représentée par la liste chaînée suivante :



DÉFINITIONS D'UNE LISTE CHAÎNÉE

Définition récursive

- Une liste chaînée est :
 - soit une liste vide
 - soit composée d'une cellule (la première) chaînée à une sous liste (une sous liste est une liste obtenue après suppression de la première cellule).
- Remarque**
- Dans une liste chaînée, chaque élément est lié à son successeur et il n'est donc pas possible d'accéder directement à un élément quelconque de la liste.

DÉCLARATION D'UNE LISTE CHAÎNÉE

Déclaration algorithmique

- Type
- **Cellule** = Enregistrement (Structure)
 - Début
 - `donnee : Telement`
 - `suivant : Adresse d'une cellule`
 - Fin
- **Liste** = Adresse d'une cellule
- Une adresse s'appelle aussi un **pointeur**.

DÉCLARATION D'UNE LISTE CHAÎNÉE

Accès aux champs d'une structure

- Si *s* est une structure (enregistrement) composées de plusieurs champs nommés *a*, *b* etc.
 - Alors *s.a* est la variable correspondant au membre *a* de la structure.
- Si *p* est un pointeur sur un enregistrement à plusieurs champs nommés *a*, *b* etc.
 - Alors *p.a* est la variable correspondant au champ *a*.
- Un pointeur qui ne pointe sur rien a la valeur nul.

DÉCLARATION D'UNE LISTE CHAÎNÉE

Déclaration en langage C

VERSION 1

```
struct cellule{
    Telement donnee;

    struct cellule *suivant;
};

typedef struct cellule *Liste;
```

VERSION 2

```
typedef struct cellule{
    Telement donnee;

    struct cellule *suivant;
}Cellule;

typedef Cellule *Liste;
```

DÉCLARATION D'UNE LISTE CHAÎNÉE

Déclaration en langage C

VERSION 3

```
typedef struct cellule{
    Telement donnee;

    struct cellule *suivant;
} *Liste;
```

VERSION 4

```
typedef struct cellule * Liste;

struct cellule{
    Telement donnee;

    Liste suivant;
};
```

DÉCLARATION D'UNE LISTE CHAÎNÉE

Accès aux membres d'une structure non pointée

- On utilise l'opérateur de sélection (.) pour sélectionner chaque membre de la structure.
- **Exemple**
- `struct cellule cel` ↔ `Cellule cel ;`
- Si `cel` est une variable de type `cellule` alors
 - `cel.donnee` désigne le membre donnée de la structure `cellule`
 - `cel.suivant` désigne le membre suivant de la structure `cellule`

DÉCLARATION D'UNE LISTE CHAÎNÉE

Accès aux membres d'une structure pointée

- En langage C, on utilise l'opérateur `→` pour sélectionner les différents membres d'une structure pointée.
- **Exemple**
- `struct cellule *cel` ↔ `Cellule *cel ;` ↔ `Liste l;`
- Si `l` est une variable de type `Liste` alors
 - `l→donnee` ↔ `(*l).donnee` : désigne le membre donnée de la structure `Liste`
 - `l→suivant` ↔ `(*l).suivant` : désigne le membre suivant de la structure `Liste`

VALEUR D'ADRESSE NULL

- La valeur NULL ne correspond à aucune adresse possible de l'objet.
- **Affectation :**
 - $L \leftarrow \text{NULL}$ ($L = \text{NULL}$)
- **NULL :**

 - - permet l'initialisation d'une structure dynamique.
 - - permet en particulier de tester les limites d'une structure dynamique,
- **Le contenu d'une variable de type pointeur n'est défini que s'il est différent de NULL.**

REMARQUES

- Dans une liste, il y a autant de cellules que d'éléments.
- Le membre suivant d'une cellule contient l'adresse de la cellule suivante.
- Le membre suivant de la dernière cellule contient la valeur NULL.
- Il suffit de connaître l'adresse de la première cellule pour avoir accès à tous les éléments de la liste.
- **Une liste chaînée est définie par l'adresse de sa première cellule.**

PRIMITIVES DE BASE

- **Donnée** : retourne la donnée d'une cellule
- Fonction donnee : Liste $\rightarrow$ TElement
- **Précondition** : non est vide liste

```

Fonction donnee(l)  $\rightarrow$  TElement
P.F
    l : Liste (E )
Début
    renvoyer l.donnee
FIN
  
```

PRIMITIVES DE BASE

- **Suivant** : retourne la cellule suivante
- Fonction suivant : Liste $\rightarrow$ Liste
- **Précondition** : non est vide liste

```

Fonction suivant(l)  $\rightarrow$  Liste
P.F
    l : Liste (E )
Début
    renvoyer l.suivant
FIN
  
```

OPÉRATIONS DE BASE

- init : initialisation d'une liste à vide,
- estVide : teste si une liste donnée est vide ou pas,
- estPein : teste si une liste donnée est plein ou pas,
- parcours : parcours une liste,
- longueur : calcul la longueur d'une liste = nombre de cellules,
- inserTete : insère un élément en tête de liste,
- inserQueue : insère un élément en queue,
- accèsPos : accès par position ou par élément,
- rechElt : recherche un élément dans une liste,
- suppTete : suppression en tête dans une liste non vide,
- etc

INITIALISATION D'UNE LISTE À VIDE

Fonction *initListe* : $[] \rightarrow \text{Liste}$

Fonction initListe() → Liste

Debut

renvoyer NULL

Fin

Liste initListe(void){

return null ;

}

OPERATION EST VIDE LISTE

Fonction *estVide* : Liste --> Boolean

- Pour savoir si une liste est vide, il suffit de tester si sa première cellule vaut NULL.

Fonction estVide(l) → Boolean

P.F l : Liste (E)

Début

renvoyer l=NULL

Fin

int videListe(Liste l){

return l==null ;

}

OPERATION EST VIDE LISTE

Fonction *estPlein* : Liste --> Boolean

On suppose qu'une liste n'est jamais plein

Fonction estPlein(l) → Boolean

P.F l : Liste (E)

Début

renvoyer faux

Fin

int estPlein(Liste l){

return 0 ;

}

PARCOURS D'UNE LISTE

Le parcours d'une liste est le traitement de chaque cellule (donnée) de la liste

- **Schéma itératif**
- Pour l'écriture des algorithmes sous forme itératif, on applique la définition suivante d'une liste chaînée :
- *Une liste chaînée est constitué d'un ensemble fini de cellules (maillons) chaînées entre elles.*
- *L'adresse de la première cellule est l'adresse de la liste.*

PARCOURS D'UNE LISTE

- **version itérative**

procédure itParcours (l)

P.F l : Liste (E)

Début

Tantque non estVide(l) faire
 traiter(donnee(l))
 l ← suivant(l)

fintq

Fin

PARCOURS D'UNE LISTE

Le parcours d'une liste est le traitement de chaque cellule (donnée) de la liste

- **Schéma récursif**
- Pour l'écriture des algorithmes sous forme récursive, on applique la définition suivante d'une liste chaînée :
- **Une liste chaînée est :**
 - soit une liste vide
 - soit composée d'une cellule (la première) chaînée à une sous liste (obtenue après suppression de la première cellule) et la sous liste est aussi une liste chaînée.

PARCOURS D'UNE LISTE

- **Première version récursive**
- si la liste est vide
 - → le parcours est terminé
- la liste n'est pas vide
 - → Traiter la première cellule
 - → Effectuer le parcours de la sous liste

PARCOURS D'UNE LISTE

- Première version recursive terminale

```

procédure re1Parcours (l)
P.F  l : Liste (E)
Début
    Si non estVide(l) alors
        traiter ( donnee(l) )
        re1Parcours(suivant(l) )
    Finsi
Fin
  
```

- Remarque : Ce parcours traite les cellules de la liste de gauche à droite.

PARCOURS D'UNE LISTE

- Deuxième version récursive
 - si la liste est vide
 - → le parcours est terminé
 - la liste n'est pas vide
 - → Effectuer le parcours de la sous liste
 - → Traiter la première cellule

PARCOURS D'UNE LISTE

- Deuxième version recursive non terminale

procédure re2Parcours (l)

P.F l : Liste (E)

Début

Si non estVide(l) alors
 re2Parcours(suivant(l))
 traiter (donnee(l))

Finsi

Fin

- Remarque : Ce parcours traite les cellules de la liste de droite à gauche.

LONGUEUR D'UNE LISTE

- Longueur d'une liste = nombre de cellules
- *Fonction longueur : Liste $\rightarrow$ Entier*
- Schéma itératif
- La liste est vide
- $\Rightarrow$ L'algorithme est terminé : la liste contient 0 éléments : $lg \leftarrow 0$
- la liste n'est pas vide
 - $\Rightarrow$ La liste à au moins un élément
 - $\Rightarrow lg \leftarrow lg + 1$ et $l \leftarrow l.suivant$
 - $\Rightarrow$ Itération : tq non videListe(l) faire

LONGUEUR D'UNE LISTE

- version itérative

```

fonction itLongueur (l) → Entier
P.F  l : Liste (E)
Début
    lg ← 0
    Tantque non estVide(l) faire
        lg ← lg + 1
        l ← suivant(l)
    fintq
    renvoyer lg
Fin
  
```

©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

33

LONGUEUR D'UNE LISTE

- Longueur d'une liste = nombre de cellules
- *Fonction longueur : Liste → Entier*
- **Schéma récursif**
- La liste est vide
- ➡ L'algorithme est terminé : la liste contient 0 éléments : *renvoyer 0*
- la liste n'est pas vide
 - ➡ Elle est donc composée d'une cellule (le premier élément) chaînée à une sous liste (suivant(l)). La longueur de la liste est donc égale à 1 plus la longueur de la sous liste
 - ➡ on exécute l'action : renvoyer *1 + longueur(suivant(l))*

©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

34

LONGUEUR D'UNE LISTE

- **version récursive**

fonction reLongueur (l) → Entier

P.F l : Liste (E)

Début

si estVide(l) alors

renvoyer 0

sinon

renvoyer 1 + reLongueur(suivant(l))

finsi

Fin

35

ACCÈS DANS UNE LISTE

- On distingue deux sortes d'accès :

- 1) l'accès par position,
- 2) l'accès associatif.

- **Accès par position**

- Accès à la $k^{\text{ième}}$ position dans une liste chaînée quelconque.

- *Fonction accesPosition : Entier X Liste → Booléenne (ou TElement ou Adresse)*

ACCÈS PAR POSITION : VERSION ITÉRATIVE

- **Fonction itAccesPositionListe(pos, l) → booléenne**
- P.F : pos : Entier (E) l : Liste (E)
- **Début**
 - $i \leftarrow 1$
 - Tanque ($i < \text{pos}$ et non estVide(l)) faire
 - $i \leftarrow i + 1$
 - $l \leftarrow \text{suivant}(l)$
 - Fintq
 - Renvoyer ($i = \text{pos}$ et non estVide(l))
 - *Autres solutions Renvoyer non (estVide(l))*
- **Fin**

ACCÈS PAR POSITION : VERSION ITÉRATIVE

- Cette fonction retourne l'adresse du $k^{\text{ième}}$ élément s'il existe et retourne NULL sinon.

• **Fonction accesPosition : Entier X Liste → Liste**

- **Fonction itAccesPositionListe(pos, l) → Liste**
- P.F : pos : Entier (E) l : Liste (E)
- **Début**
 - Tanque ($\text{pos} > 1$ et non estVide(l)) faire
 - $\text{pos} \leftarrow \text{pos} - 1$
 - $l \leftarrow \text{suivant}(l)$
 - Fintq
 - Renvoyer l
- **Fin**

ACCÈS PAR POSITION : VERSION RÉCURSIVE

Fonction accesPosition : Entier X Liste → Booléenne

si la liste est vide

- → Arrêt de l'algorithme : renvoyer Faux

si la liste n'est pas vide

si pos=1 → Arrêt de l'algorithme : renvoyer Vrai

sinon : la liste est non vide et pos > 1

- → On exécute l'action : renvoyer (pos-1, l.suivant)

ACCÈS PAR POSITION : VERSION RÉCURSIVE

- **Fonction reAccesPositionListe(pos, l) → booléenne**
- P.F : pos : Entier (E) l : Liste (E)
- **Début**
 - Si (estVide(l)) alors**
 - Renvoyer Faux**
 - Sinon**
 - Si pos=1 alors**
 - Renvoyer Vrai**
 - Sinon**
 - renvoyer reAccesPositionListe(pos-1, suivant(l))**
 - Finsi**
 - Finsi**
- **Fin**

ACCÈS PAR POSITION : VERSION RÉCURSIVE

- Cette fonction retourne l'adresse du $k^{\text{ième}}$ élément s'il existe et retourne NULL sinon.

• *Fonction accesPosition : Entier X Liste $\rightarrow$ Liste*

- **Fonction reAccesPositionListe(pos, l) $\rightarrow$ Liste**
- P.F : pos : Entier (E) l : Liste (E)
- **Début**
 - Si estVide(l) ou (pos=1) alors
renvoyer l
 - Sinon
renvoyer reAccesPositionListe(pos-1, suivant(l))
 - Finsi
- **Fin**

ACCÈS ASSOCIATIF : VERSION ITÉRATIVE

Fonction rechElt : TElement X Liste $\rightarrow$ Booléenne (ou Liste)

Fonction itRechElt(elt, l) $\rightarrow$ Booléenne
P.F elt : TElement (E) l : Liste (E)
Début
 Tanque (non estVide(l) et donnee(l) $\neq$ elt) faire
 l $\leftarrow$ suivant(l)
 Fintq
 Renvoyer non estVide(l)
Fin

ACCÈS ASSOCIATIF : VERSION ITÉRATIVE

Fonction rechElt : TElement X Liste--> Booléenne (ou Liste)

Fonction itRechElt(elt, l)→ Liste

P.F **elt : Telement (E) l : Liste (E)**

Début

Tanque (non estVide(l) et donnee(l)≠elt) faire
 l ← suivant(l)

Fintq

Renvoyer l

Fin

ACCÈS ASSOCIATIF : VERSION RÉCURSIVE

Fonction rechElt : TElement X Liste--> Booléenne (ou Liste)

Fonction reRechElt(elt, l)→ Booléene

P.F **elt : TElement (E) l : Liste (E)**

Début

Si estVide(l) alors

Renvoyer Faux

Sinon si donnee(l)=elt alors

RenvoyerVrai

Sinon

Renvoyer reRechElt(elt, suivant(l))

Finsi

Finsi

Fin

ACCÈS ASSOCIATIF : VERSION RÉCURSIVE

Fonction rechElt : TElement X Liste--> Booléenne (ou Liste)

Fonction reRechElt(elt, l)→ Liste

P.F **elt : TElement (E) l : Liste (E)**

Début

Si estVide(l) ou donnee(l)=elt alors

Renvoyer l

Sinon

Renvoyer reRechElt(elt, suivant(l))

Finsi

Fin

RECHERCHE D' UN ÉLÉMENT DANS UNE LISTE TRIÉE

Fonction rechEltListTrie : TElement X Liste--> Booléenne (ou Liste)

- **Définition d'une liste triée**
- Une liste vide est triée
- Une liste composée d'un seul élément est triée
- Une liste de plus d'un élément est triée si tous les éléments consécutifs vérifient la relation d'ordre ($\leq$) suivante :
 - $l \neq \text{NULL}$ et $\text{suivant}(l) \neq \text{NULL}$ et $\text{donnee}(l) \leq \text{suivant}(\text{donnee}(l))$

RECHERCHE D' UN ÉLÉMENT DANS UNE LISTE TRIÉE

Fonction rechEltListTrie : TElement X Liste--> Booléenne (ou Liste)

- **Version itérative**

voir TD

RECHERCHE D' UN ÉLÉMENT DANS UNE LISTE TRIÉE

Fonction rechEltListTrie : TElement X Liste--> Booléenne (ou Liste)

- **Version récursive**

```

• Fonction reRechEltListTrie(elt, l)→ Booléenne
• P.F          elt : TElement (E )   l : Liste (E )
• Début
    Si estVide(l) ou donnee(l) > elt alors
        Renvoyer Faux
    Sinon si donnee(l) = elt alors
        Renvoyer Vrai
    sinon
        Renvoyer reRechEltListTrie(elt, suivant(l) )
    Fsi
  Fsi
Fin

```


ALGORITHMES D'INSERTION DANS UNE LISTE

• Insertion d'un élément en tête de liste

- C'est un algorithme **très important** que nous utiliserons très souvent pour réaliser les autres insertions.
- Cette insertion est particulière puisqu'elle modifie l'adresse de la liste.

• *inserTête : TElement X Liste --> Liste*

INSERTION EN TÊTE

• Principe

- ❖ Création d'une nouvelle cellule

$cel \leftarrow \text{allocMemCellule}()$



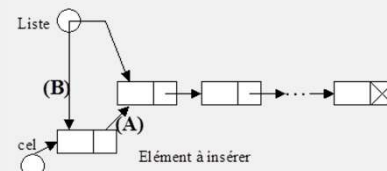
- ❖ Remplissage du membre donnée de la cellule

$cel.donnee \leftarrow elt$



- ❖ Réalisation des chaînages (A) et (B) dans cet ordre :

- $Cel.suivant \leftarrow l$ /* Chaînage A */
- $l \leftarrow cel$ /* Chaînage B */



INSERTION EN TÊTE

Fonction inserTete : TElement X Liste--> Liste

- **Fonction inserTete(elt, l)→ Liste**
- **P.F** elt : TElement (E) l : Liste (E)
- **Début**

cel ← allocMemCellule() //alloc Memoire

cel.donnee ← elt //Remplissage

cel.suivant← l // Liaison A

l← cel // Liaison B

renvoyer l

Fin

©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

51

INSERTION EN TÊTE

Fonction inserTete : TElement X Liste--> Liste

#include <stdlib.h>

```
Liste inserTete(TElement elt, Liste l){
Liste cel ;
    cel=(Liste) malloc(sizeof(struct cellule)) ;
    cel->donnee=elt ;
    cel->suivant=l ;
    l=cel ;
    return l ;
}
```

©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

52

INSERTION EN TÊTE

Fonction inserTete : TElement X Liste--> Liste

- **procédure inserTete(elt, l)**
- **P.F** elt : TElement (E) l : Liste (E/S)
- **Début**

```

cel ← allocMemCellule()    //alloc Memoire

cel.donnee ← elt            //Remplissage

cel.suivant ← l            // Liaison A

l ← cel                    // Liaison B

Fin
```

INSERTION EN TÊTE

Fonction inserTete : TElement X Liste--> Liste

#include <stdlib.h>

```

void inserTête(TElement elt, Liste *l){
Liste cel ;
    cel=(Liste) malloc(sizeof(struct cellule)) ;
    cel→donnee = elt ;
    cel→suivant = *l ;
    *l = cel ;
}
```

INSERTION EN QUEUE

Fonction inserQueue : TElement X Liste--> Liste

- **Schéma itératif**

- ❖ si la liste est vide, on est ramené à une insertion en tête de liste.

- ❖ On procède comme suit si la liste n'est pas vide :

- ❖ Création d'une nouvelle cellule

$cel \leftarrow \text{allocMemCellule}()$

- ❖ Remplissage du membre donnée de la cellule

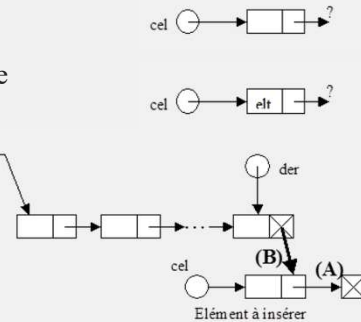
$cel.donnee \leftarrow elt$

- ❖ Réalisation des chaînages (A) et (B) :

- $cel.suivant \leftarrow l$ /* Chaînage A */

- $der.suivant \leftarrow cel$ /* Chaînage B */

- Il faut connaître l'adresse de la dernière cellule « der ».



©B. DERDOURI

CM Listes Chaînées

Module Algo2 L2-MI_Sem3-U-CERGY

9/9/2020

55

INSERTION EN QUEUE

Fonction inserQueue : TElement X Liste--> Liste

- on suppose que **dernier** est une fonction qui renvoie la dernière cellule d'une liste.

Procédure inserQueue(elt, l)

P.F **Elt : TElement (E)** **l : liste (E/S)**

Début

Si estVide(l) alors
 inserTete(elt, l)

Sinon

$der \leftarrow \text{dernier}(l)$

(1) cel ← allocMemCellule()

 // Alloc Memoire

(2) cel.donnee ← elt

 // Remplissage

(3) cel.suivant ← NULL // ou suivant(der)

 // Liaison A

(4) der.suivant ← cel

 // Liaison B

Finsi

Fin

INSERTION EN QUEUE

Fonction inserQueue : TElement X Liste --> Liste

Remarque : Les actions (1), (2), (3) et (4) correspondent à l'appel suivant :

`inserTete(elt, der.suivant)`

Procédure `inserQueue(elt, l)`

P.F **Elt :** TElement (E)

l : liste (E/S)

Début

Si estVide(l) **alors**
 inserTete(elt, l)

Sinon

der ← **dernier**(l)
 inserTete(elt, **suivant**(der))

Finsi *// Attention traduction en C*

Fin

INSERTION EN QUEUE : VERSION RÉCURSIVE

Fonction inserQueue : TElement X Liste --> Liste

- **Raisonnement**

- si la liste est vide

- → on insère l'élément en tête : `inserTete(elt, l)`

- si la liste n'est pas vide

- → On insère l'élément en fin de la sous liste de l :

- `inserQueue(elt, suivant(l))`

INSERTION EN QUEUE : VERSION RÉCURSIVE

Fonction inserQueue : TElement X Liste--> Liste

```

Procédure inserQueue(elt, l)
P.F      Elt : T_elémen (E)
         l : liste (E/S)
Début
    Si estVide(l) alors
        inserTete(elt, l)
    Sinon
        inserQueue(elt, suivant(der))
Finsi    //Attention traduction en C
Fin
  
```

©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

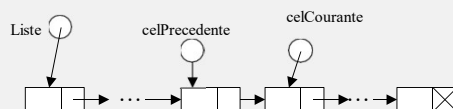
59

ALGORITHME DE LA FONCTION DERNIER

Fonction dernier : Liste--> Liste

Hypothèse : on suppose que la liste est non vide

- **Version itérative**
- On effectue un parcours de tous les éléments de la liste en préservant à chaque fois l'adresse de la cellule précédente.



©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

60

FONCTION DERNIER: VERSION 1 ITÉRATIVE

Fonction dernier(l) → Liste (=Adresse de la dernière cellule)

P.F L : Liste (E)

Début

celCourante ← l

Tantque non estVide(celCourante) faire

CelPrecedente ← celCourante

CelCourante ← Suivant(celCourante)

Fintq

Renvoyer CelPrecedente

Fin

©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

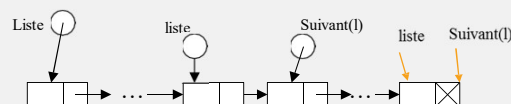
61

VERSION 2 ITÉRATIVE : FONCTION DERNIER

Fonction dernier : Liste → Liste

Hypothèse : on suppose que la liste est non vide

- **Version 2 itérative**
- Cette deuxième version consiste à ne pas utiliser la variable auxiliaire précédent mais à noter que : l et suivant(l) contiennent les adresses de deux cellules consécutifs.
- Il suffit de parcourir la liste en s'arrêtant quand la liste est réduite à une seule cellule qui est la dernière c'est à dire suivant(l) est vide.



©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

62

FONCTION DERNIER: VERSION 2 ITÉRATIVE

Fonction dernier(l) → Liste (=Adresse de la dernière cellule)

P.F L : Liste (E)

Début

der ← l

Tantque non estVide(suivant(der)) faire

der ← suivant(der)

Fintq

Renvoyer der

Fin

VERSION RÉCURSIVE : FONCTION DERNIER

Fonction dernier : Liste--> Liste

Hypothèse : on suppose que la liste est non vide

- **Raisonnement**

- ❖ si la liste est réduite à une seule cellule (c-à-d suivant(l) est vide)

- → Terminé : la liste l contient l'adresse de la dernière cellule

- → Renvoyer l

- ❖ si la liste est composée de plusieurs cellules :

- la dernière cellule se trouve dans la sous liste

- → Renvoyer dernier(suivant(l))

FONCTION DERNIER: VERSION RÉCURSIVE

Fonction dernier : Liste--> Liste

Hypothèse : on suppose que la liste est non vide

Fonction dernier(l) → Liste (=Adresse de la dernière cellule)

P.F.L : Liste (E)

Début

Si estVide(suivant(l)) alors

renvoyer l

Sinon

renvoyer dernier(suivant(l))

Finsi

Fin

INSERTION DANS UNE LISTE TRIÉE

Fonction inserEltListTrie : TElement x Liste--> Liste

- **Version récursive**

- ❖ Si la liste est vide

- insertion en tête de liste

- ❖ Si l'élément à insérer est inférieur ou égal à la donnée de la première cellule

- on effectue une insertion en tête de liste

- ❖ Si l'élément à insérer est supérieur à la donnée de la première cellule

- On effectue l'insertion par rapport à la sous liste.

INSERTION DANS UNE LISTE TRIÉE

```

procedure inserEltListTrie (elt, l)
P.F elt : TElement (E)   l : Liste (E/S)
Debut
    Si (estVide(l)) ou (elt <= donnee(l) ) alors
        inserTete(elt , l) // appel proc inserTete
    Sinon
        inserEltListTrie (elt, suivant(l))
    Finsi
Fin
  
```

Req : Attention à la traduction en C

INSERTION DANS UNE LISTE TRIÉE

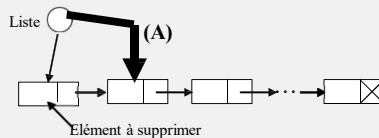
Version itérative

Voir TD

SUPPRESSION ELÉMENT DANS UNE LISTE QUELCONQUE

Hypothèse : on suppose que la liste est non vide et aucun ordre sur les éléments de la liste.

Suppression du premier élément ↔ **Suppression en tête**



pour libérer la mémoire occupée par la première cellule, il faut donc préserver l'adresse de la tête de la liste avant la modification du chaînage (A)

SUPPRESSION EN TÊTE

Procédure suppTete(l)

P.F 1 : Liste (E/S)

Début

$ll \leftarrow l$ //Préservation de l'adresse de la première cellule

$l \leftarrow \text{suivant}(l)$ // Chaînage A

$\text{libMem}(ll)$ // Libération de la mémoire occupé par ll

Fin

SUPPRESSION EN TÊTE

Fonction $\text{suppTete}(l) \rightarrow \text{Liste}$

P.F $l : \text{Liste (E)}$

Début

$ll \leftarrow l$ //Préservation de l'adresse de la première cellule

$l \leftarrow \text{suivant}(l)$ // Chaînage A

$\text{libMem}(ll)$ // Libération de la mémoire occupée par ll

renvoyer l

Fin

71

SUPPRESSION EN TÊTE

```
void suppTete(Liste *l){
```

```
Liste ll ;
```

```
    ll = *l ;
```

```
    *l = suivant(*l) ;
```

```
    free(ll) ;
```

```
}
```

```
Liste suppTete(Liste l){
```

```
Liste ll ;
```

```
    ll = l ;
```

```
    l = suivant(l) ;
```

```
    free(ll) ;
```

```
    return l ;
```

```
}
```

72

DESTRUCTION D'UNE LISTE

Procédure ItSuppListe(l) // Version itérative

P.F l : Liste (E/S)

Début

Tantque non estVide(l) faire

 suppTete(l) // appel proc

finTantque

Fin

DESTRUCTION D'UNE LISTE

Procédure reSuppListe(l) // Version récursive

P.F l : Liste (E/S)

Début

Si non estVide(l) faire

 suppTete(l) // appel proc

 reSuppListe(l)

finSi

Fin

SUPPRESSION ELÉMENT DANS UNE LISTE QUELCONQUE

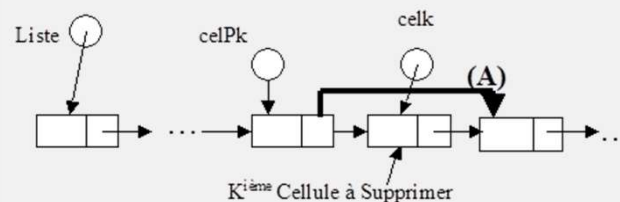
Hypothèse : la liste est non vide

L'élément à supprimer peut être déterminé par sa **position** ou par sa **valeur**.

- **Suppression par position**
- On souhaite supprimer la $k^{\text{ième}}$ cellule d'une liste.

SUPPRESSION PAR POSITION

- **Schéma itératif**
- Il faut déterminer l'adresse précédent de la cellule qui précède celle que l'on veut supprimer : c'est à dire l'adresse de la $(k-1)^{\text{ième}}$ cellule. Si la $k^{\text{ième}}$ cellule existe, on modifié le chaînage suivant comme suit :



SUPPRESSION PAR POSITION : VERSION ITÉRATIVE

Précondition : non estVide(l)

Fonction itSuppPosListe(k, l) → Booléenne

P.F k : Entier (E) l : Liste (E/S)

Début

Si (k=1) alors

suppTete(l)

Renvoyer vrai

Sinon

supp ← faux

celPk ← accesPositionListe(k-1, l)

SUPPRESSION PAR POSITION : VERSION ITÉRATIVE

Si non estVide(celPk) alors

celK ← suivant(celPk)

Si non estVide(celK) alors

Supp ← vrai

celPK.Suivant ← suivant(celK)

LibMém(celK)

Finsi

Finsi

Renvoyer supp

Finsi

Fin

SUPPRESSION PAR POSITION : VERSION ITÉRATIVE

Précondition : non estVide(l)

En utilisant supTete, on obtient la version suivante

Fonction itSuppPosListe(k, l) → Booléenne

P.F k : Entier (E) l : Liste (E/S)

Début

Si (k=1) alors

supTete(l)

Renvoyer vrai

Sinon

supp ← faux

celPk ← accesPositionListe(k-1, l)

©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

79

SUPPRESSION PAR POSITION : VERSION ITÉRATIVE

Si non estVide(celPk) alors

celK ← suivant(celPK)

Si non estVide(celK) alors

Supp ← vrai

supTete(suivant(celPK))

//(1)=celPK.Suivant←suivant(celK)

//(2)=LibMém(celK)

// 1 et 2 = supTete

// Attention traduction C

Finsi

Finsi

Renvoyer supp

Finsi

Fin

©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

80

SUPPRESSION PAR POSITION : VERSION RÉCURSIVE

- **Schéma récursif**

- ❖ liste est vide → Suppression est impossible ↔ // Cas d'arrêt

- ❖ liste est non vide

- * $k=1$ → suppression en tête de ↔

liste appel algo `suppTete(l)`

- * $k>1$ → suppression du $(k-1)$ ième élément par rapport à la sous liste

- ↔ appel récursif **`suppPosListe(k-1, suivant(l))`**

SUPPRESSION PAR POSITION : VERSION RÉCURSIVE

Fonction `reSuppPosListe(k, l) → Booléenne`

P.F `k : Entier (E)    l : Liste (E/S)`

Début

Si `(estVide(l))` alors

Renvoyer Faux

Sinon

Si $k=1$ alors

`SuppTete(l)`

Renvoyer Vrai

Sinon

Renvoyer `reSuppPosListe(k-1, suivant (l) )`

Finsi

Finsi

Fin

// attention traduction en C

SUPPRESSION PAR VALEUR : VERSION ITÉRATIVE

- Schéma itératif
- Raisonement voir TD

©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

83

SUPPRESSION PAR VALEUR : VERSION ITÉRATIVE

Fonction itSuppValListe(k, l) → Booléenne

P.F val : TElement (E) l : Liste (E/S)

Début

Fin

// attention traduction en C

©B. DERDOURI

CM Listes Chainées

Module Algo2 L2-M1_Sem3-U-CERGY

9/9/2020

84

SUPPRESSION PAR VALEUR : VERSION RÉCURSIVE

• Schéma récursif

❖ Si La Liste est vide → Suppression est impossible ↔ // Cas d'arrêt

❖ Si la liste est non vide →

* Si l'élément à supprimer est égal à la donnée de la première cellule
suppression → suppression en tête de liste ↔ appel algo supTete

* Sinon → la suppression s'effectue par rapport à la sous liste

↔ *l'appel récursif*

85

SUPPRESSION PAR VALEUR : VERSION RÉCURSIVE

Fonction reSuppValListe(val, l) → Booléenne

P.F val : TElement (E) l : Liste (E/S)

Début

Si estVide(l) alors

Renvoyer Faux

Sinon

Si donnee(l) =val alors

supTete(l) // appel proc

Renvoyer Vrai

Sinon

Renvoyer reSuppValListe(val, suivant(l))

Finsi

Finsi

Fin

// attention traduction en C

AVANTAGES ET INCONVÉNIENTS

Avantages

- Gestion dynamique (aucune longueur n'est imposée)
- Traite aisément les opérations sur les listes :
 - Parcours séquentiel
 - Insertion d'un élément à une place quelconque
 - Suppression d'un élément à une place quelconque
 - Concaténation de deux listes
 - Etc

Par contre, dans un tableau, l'insertion (resp. Suppression) n'est possible que par un décalage de tous les éléments du tableau.

AVANTAGES ET INCONVÉNIENTS

Inconvénients

- accès séquentiels : accès au $k^{\text{ième}}$ éléments d'une liste n'est plus directe
- difficultés d'utilisation
- occupe plus de place mémoire, puisqu'il faut stocker non seulement les données mais aussi les pointeurs (adresses).
- Impossible d'effectuer la recherche dichotomique

Dans un tableau l'accès aux éléments du tableau se fait d'une manière **directe**.

Possibilité d'effectuer la recherche dichotomique

L'utilisation d'un tableau est simple.